

Variables, Locals & Outputs

Make your Terraform code dynamic and reusable — the first half of the Variables story.

Input variables

Setting values

Validation

Locals & outputs

Lab 03

Today's roadmap



1 - Input variables

Types, optional arguments, and how to declare them well.



2 - Setting values

The precedence order and why several tfvars files exist.



3 - Validation & locals

Guard inputs early; compute reusable local values.



4 - Outputs & Lab 03

Return & protect values; parameterize your project.

01

Input Variables

Parameters for your Terraform code — customize without touching the source.

Variables in Terraform



Inputs

Parameters set from outside —
`var.name`.



Locals

Computed helpers inside the
code — `local.name`.



Outputs

Values returned after apply —
`output`.



Values live **separately** from the plan, so they're easy to read and edit from a single file.

Types & optional arguments

HCL

```
# one example per type
variable "location"    { type = string }
variable "node_count"  { type = number }
variable "enabled"     { type = bool }
variable "zones"       { type = list(string) }
variable "tags"        { type = map(string) }

variable "network" {
  type = object({
    name = string
    cidr = string
  })
}
```

Every input is **typed**; four optional arguments refine it:

- **default** — makes the variable optional
- **description** — documents it
- **validation** — custom rules (next)
- **sensitive** — hides it from output



No default = you **must** supply a value. Terraform has no unassigned variables.

Setting values: precedence

💡 Terraform checks top → bottom — the **first source that sets a value wins** .

01

Command line `-var` / `-var-file`

Highest precedence — overrides everything below



02

Config files `*.tfvars`

Loaded from files on disk — the loading order is on the next slide



03

Environment variables `TF_VAR_name`

Great for secrets and CI/CD pipelines



04

Default in the `variable` block

Fallback when nothing else provides a value



05

Interactive prompt

Last resort — Terraform stops and asks you



tfvars strategy

📖 How config files load — and, when two set the same value, **the last one loaded wins** .

AUTO · 1

terraform.tfvars

Loaded automatically, first

<>

AUTO · 2

***.auto.tfvars**

Auto-loaded next, alphabetical order

<>

EXPLICIT · 3

-var-file=...

Loaded last → overrides the rest

TERRAFORM.TFVARS

```
location = "westeurope"
prefix   = "tbernardo"
```

Auto-loaded — shared defaults, no flag needed.

NETWORK.AUTO.TFVARS

```
subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
tags          = { owner = "tiago", env = "dev" }
```

Also auto-loaded, in alphabetical order.

BASH

```
terraform plan -var-file="prod.tfvars"    # explicit file — overrides the auto-loaded ones
```

Input validation

Custom rules Terraform checks at `plan` time — bad input fails **before** any resource is created.

- **condition** — an expression that must be `true`
- **error_message** — shown when the check fails
- Wrap risky calls in `can(...)` so a parse error becomes a clean failure

HCL

```
variable "prefix" {  
  type = string  
  validation {  
    condition      = can(regex("^[a-z][a-z0-9-]{2,20}$", var.prefix))  
    error_message = "3-21 chars: lowercase, digits, hyphens."  
  }  
}
```

⚠ Invalid input stops the plan — nothing is created or changed.

Common validation conditions

■ Allowed set

```
contains(["dev", "prod"], var.env)
```

■ Length

```
length(var.name) <= 20
```

■ Pattern — regex

```
can(regex("^[a-z-]+$", var.name))
```

■ Numeric range

```
var.count >= 1 && var.count <= 10
```

■ Prefix / suffix

```
startswith(var.image, "app-")
```

■ Every element

```
alltrue([for s in var.subnets : s != ""])
```

💡 Combine any of these with `&&`, `||`, `!` and the ternary `? :` to build richer rules.

02

Locals, Outputs & Interpolation

Compute once, reuse everywhere — and return what matters.

Local values

- Defined, assigned and used in the same code
- Accessed as `local.name`
- Set by **you**, not by the user (unlike inputs)
- Change a value once, in one central place

HCL

```
locals {  
  prefix = "tbernardo"  
  common_tags = {  
    owner      = "tiago"  
    managed_by = "terraform"  
  }  
}  
  
# usage  
name = "${local.prefix}-app-rg"
```

Outputs & sensitive values

- Expose values after apply, or feed other resources
- Printed automatically on the standard output
- Mark `sensitive = true` to redact secrets



`terraform output name` reveals a sensitive value on demand; `terraform show` reads anything in state.

HCL

```
output "repo_clone_url" {  
  value      = github_repository.course.http_clone_url  
  sensitive = true  
}
```

Direct access vs. interpolation

(x) Direct

Use a value on its own — no quotes, no `${...}`.

HCL

```
location = var.location
count    = var.node_count
tags     = local.common_tags
```

⚠ Don't wrap a lone value: `"${var.location}"`
→ just `var.location`.

<> Interpolation `${...}`

Embed an expression **inside** a string — compose, call functions, branch.

HCL

```
name = "${var.prefix}-${var.env}"    # compose
slug = "${lower(var.name)}-app"      # function
tier = "${var.ha ? "P1v3" : "B1"}"   # branch
```

💡 Inside `${ }` you can reference variables, run functions, and use conditionals — the gateway to **Session 04**.

 LIVE DEMO

Variables in action

Walk through `variables.tf`, locals, tfvars files and outputs — plan with `-var-file` and read an output value.

Demo · 03 Provider · GitHub

(x) LAB 03 · 30-40 MIN

Parameterize your project

Add variables, locals and outputs to your repo project; use tfvars files to drive different values and expose a clone URL.

You'll practise: variable types · tfvars · locals · sensitive outputs · `terraform output`

Key takeaways

✓ Inputs make code reusable

Typed, documented, optionally validated.

✓ Precedence matters

CLI > tfvars > env > default > prompt.

✓ Locals & outputs

Compute once; return & protect what matters.

✓ Next up

Session 04 — conditionals & iterations.



Questions?

Up next — [Session 04: Conditionals & Iterations.](#)